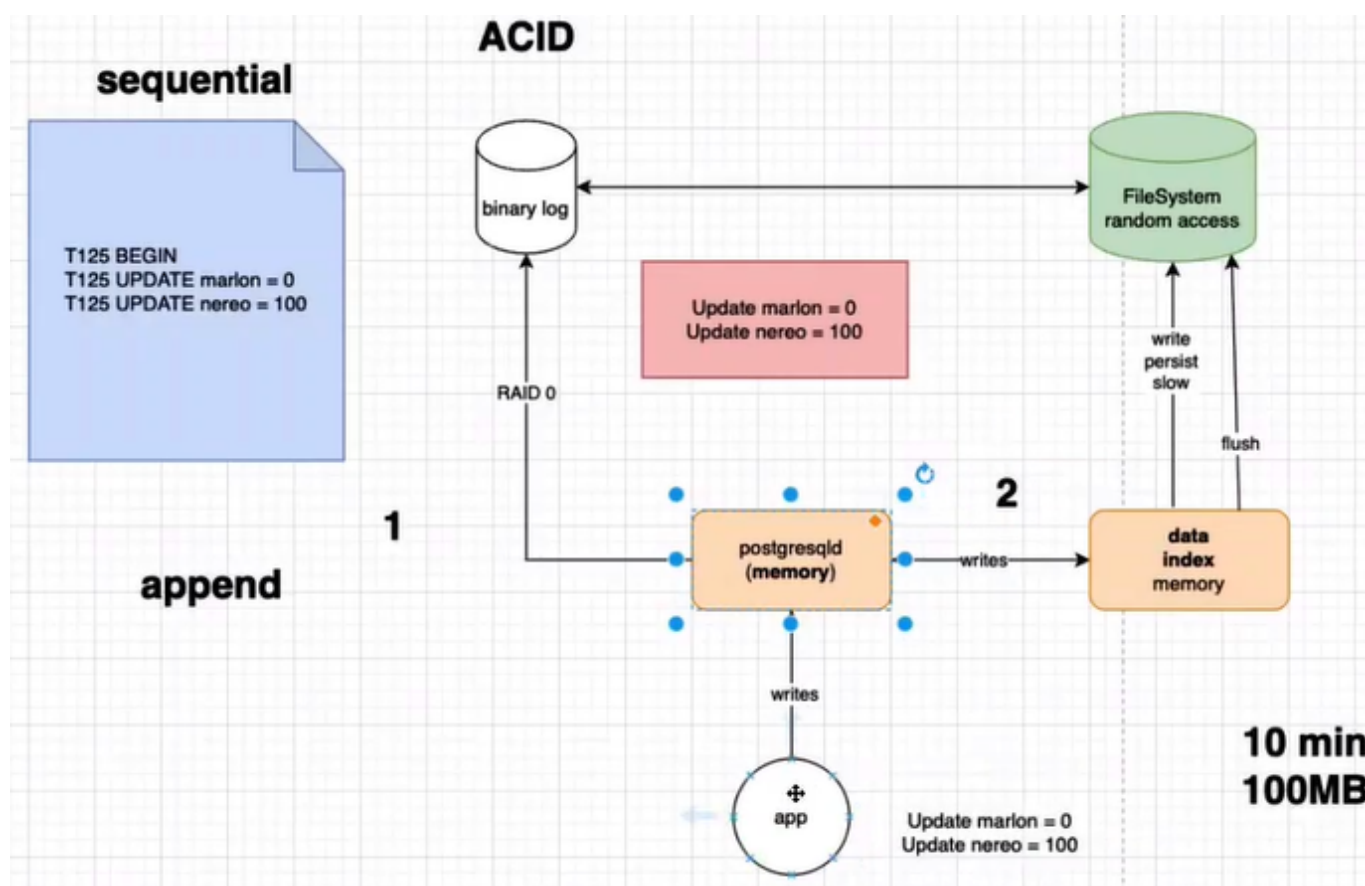


Apuntes de Clase

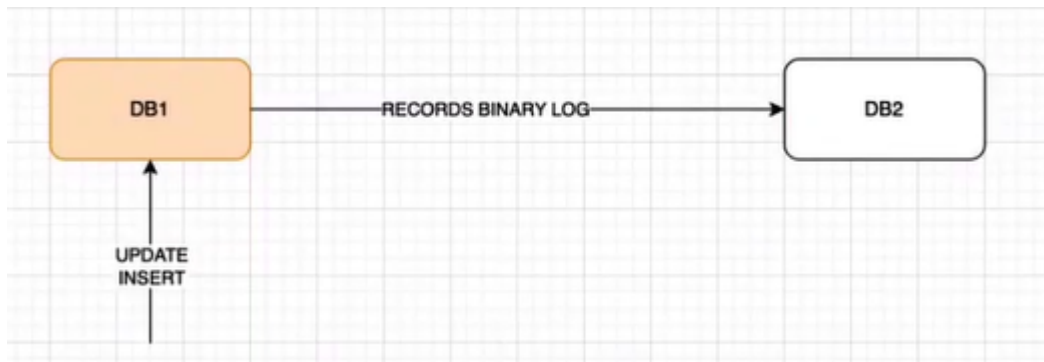
Memoria volatil: significa que los cuando el sistema crashea, se van a perder los datos que tenia almacenados en ese momento

Cuando se almacena en el **File System** hay una desventaja y es que al querer cumplir con ACID y se tiene por ejemplo tablas normalizadas, entonces se deben realizar muchas escrituras y se vuelve un proceso lento. Esto provoca que la respuesta al usuario dure bastante, por lo tanto se usa otra alternativa.

Binary log: siguiendo buenas practicas se usa en un disco por separado. Este es un archivo de acceso secuencial. Cuando entra una transaccion, se le asigna una id. Donde en el binary log va a comenzar con begin, y luego hace la instruccion. Una vez pasa este paso, se guarda en memoria principal, por lo tanto es memoria volatil. A estos datos guardados se les dice buffered. Luego, cada cierto tiempo los datos se van a persistir en el File System y una vez sucede esto, se actualiza el binary log con commit. Esto nos garantiza que los datos son persistentes en el binary log. Si los datos estando en buffered por alguna razon no logran llegar a disco, entonces lo primero que revisa es el binary log y al ver que nunca se hizo commit, ejecuta en orden secuencial las instrucciones que tenia para que esta vez si llegen al disco principal



Al tener dos bases de datos, donde una le envia los datos a la otra que es stand by, se beneficia bastante del binary log. Ya que, cuando se hace un insert, entonces esta base de datos le comparte el binary log a la otra base. De esta manera, se garantiza que aplican las mismas operaciones del binary log y en algún momento van a tener los mismos datos

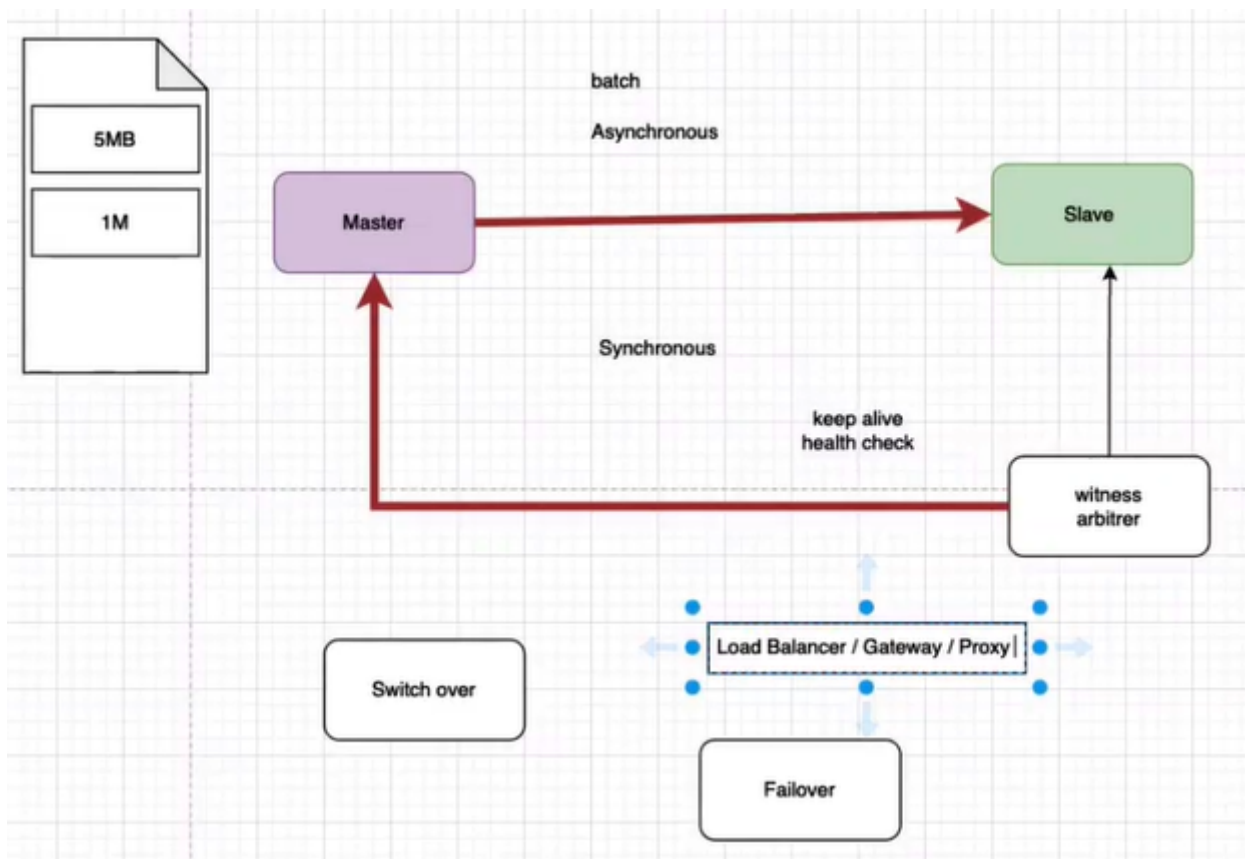


Recordar la importancia de al tener dos bases de datos, una primary y otra stand by, siempre seguir estas practicas:

- Tener el mismo poder en ambos servidores
- No tener corriendo ningun otro proceso, ya que va a quitar recursos de la computadora
- Tener observabilidad de todo lo que está sucediendo

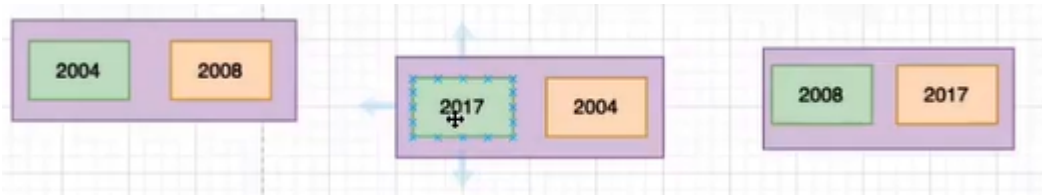
Master-Slave Replication: se tienen dos modos, asíncrono y síncrono:

- El asíncrono es por medio de batch, es decir que se va a esperar a que se llene la ventana y se envia a la base de datos Slave. Por ejemplo que en un minuto o a los 5MB se envia el binary log
- El síncrono funciona que cada vez que llega un evento al binary log esto lo envia a la base de datos Slave, lo cual gasta mas recursos. Se usa un witness, el cual esta conectado a ambas bases de datos y nos indicará si el error proviene del slave o del master. También se tiene un gateway o load balancer, los cuales se encarnan de realizar pruebas para saber si als bases están funcionando



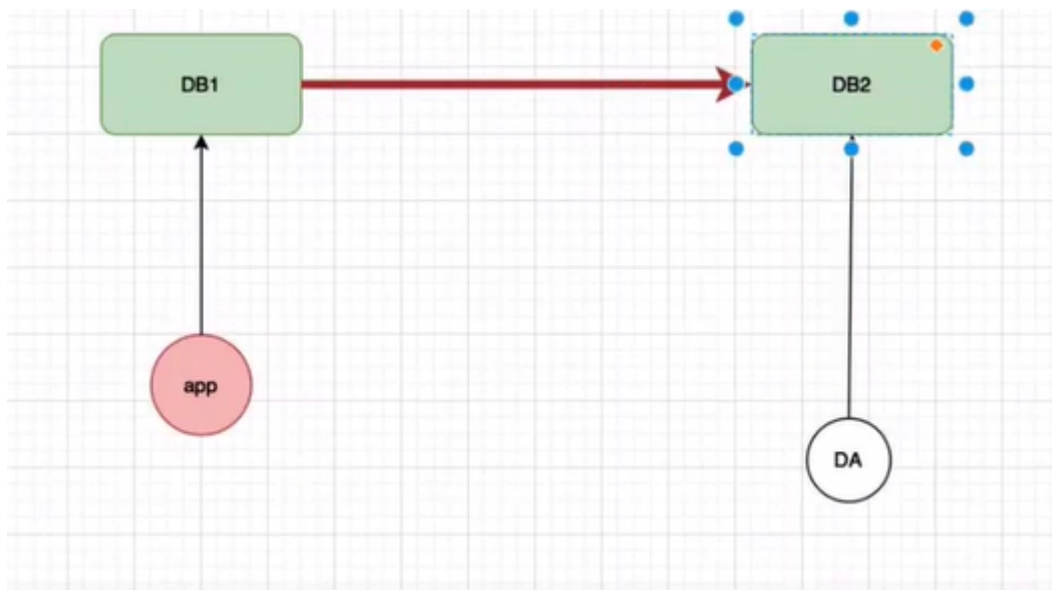
Partition: se hace partition de una tabla y estos se envian a distintos servidores, donde cada servidor tiene los datos, pero en un servidor es la primaria y en otro es el slave

El problema es que esto genera mucho **overhead**, esto significa que se gastan muchos recursos para indirectamente para su funcionalidad

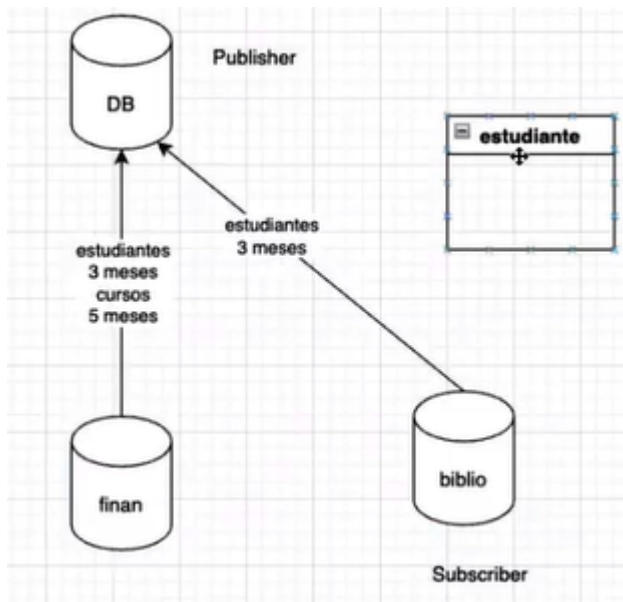


Read only replica: una base de datos que no recibe actualizaciones, pero si recibe consultas. Este tiene consistencia eventual, el cual significa que mientras se espera la ventana batch, alguien hace una consulta, el slave no recupera los ultimos datos actualizados

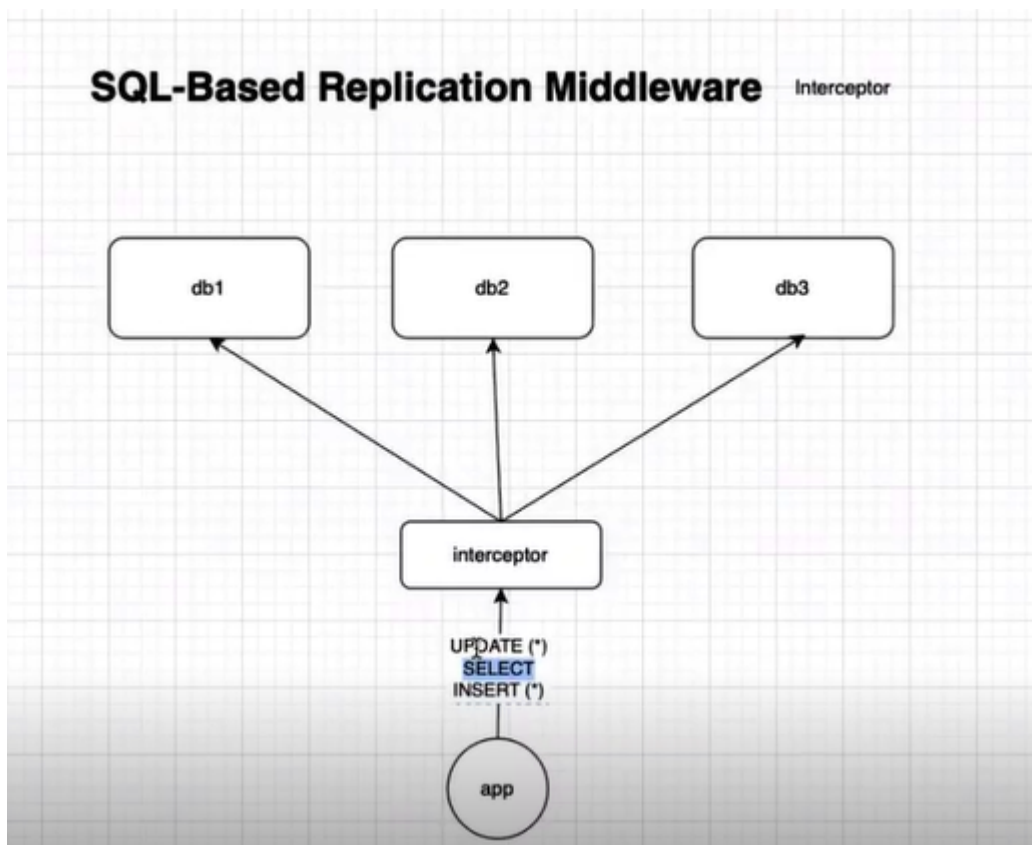
Warm Standby Using Point-In-Time Recovery: es cuando se necesita hacer alguna prueba o consultas a la base de datos, pero esta prueba afectará el rendimiento a tal punto que la base de datos fallará con la app. Por lo tanto, la solución es que se corta el vínculo de la base de datos master con la slave, de manera que se pueden hacer las pruebas a la slave que esta funcionando como primaria



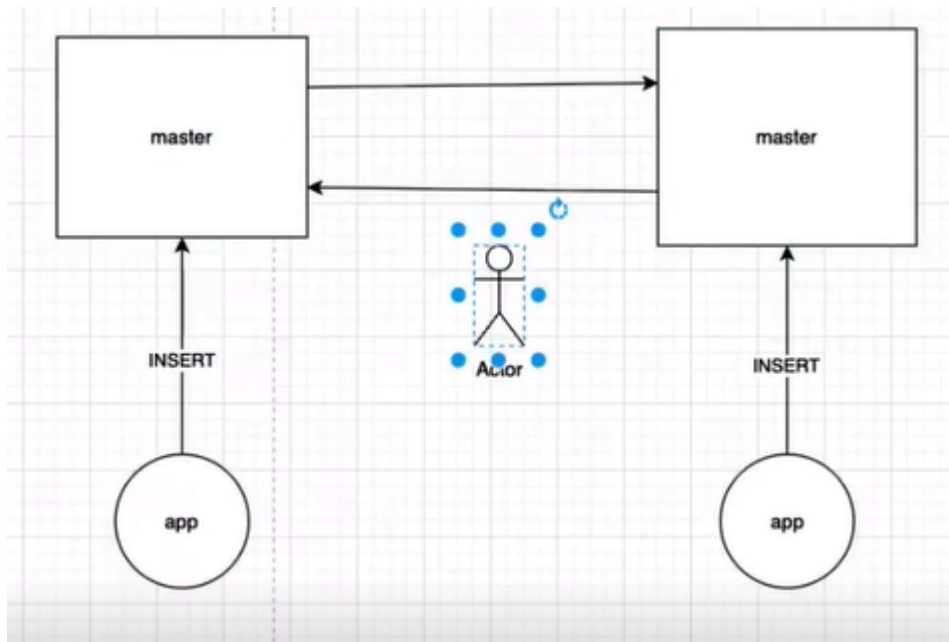
Logical Replication: se tiene una base de datos principal llamada publisher y otras bases de datos llamadas suscriber. En donde a las suscriber no les interesa todas las tablas de la publisher, por lo tanto estas consultan y actualizan las tablas necesarias cada cierto tiempo



SQL-Based Replication Middleware: se tienen bases de datos independientes, donde ninguna sabe de la existencia de la otra. También se tiene un interceptor el cual habla el language de la base de datos, cuando se tiene una operación esta se aplica a las distintas bases de datos, de modo que haya consistencia y la misma información



Asynchronous Multimaster Replication: se tienen dos bases de datos master, independientes donde cada una realiza sus operaciones. Luego de cierto tiempo, estas hacen merge entre si por medio de unos algoritmos que poseen. Pero, pueden haber conflictos y la manera de resolverlos es por medio de intervención manual de una persona



Synchronous Multimaster Replication: en este caso igual cada base de datos recibe sus propias operaciones, pero cuando hacen una misma se convierte shared resource para que no hayn conflictos. Luego se aplica un request de un servidor a otro sobre los shared resources y asi se sincronizan y obtienen los mismo datos, el problema es que los clientes deben esperar a estas etapas cuando se sincronizan

